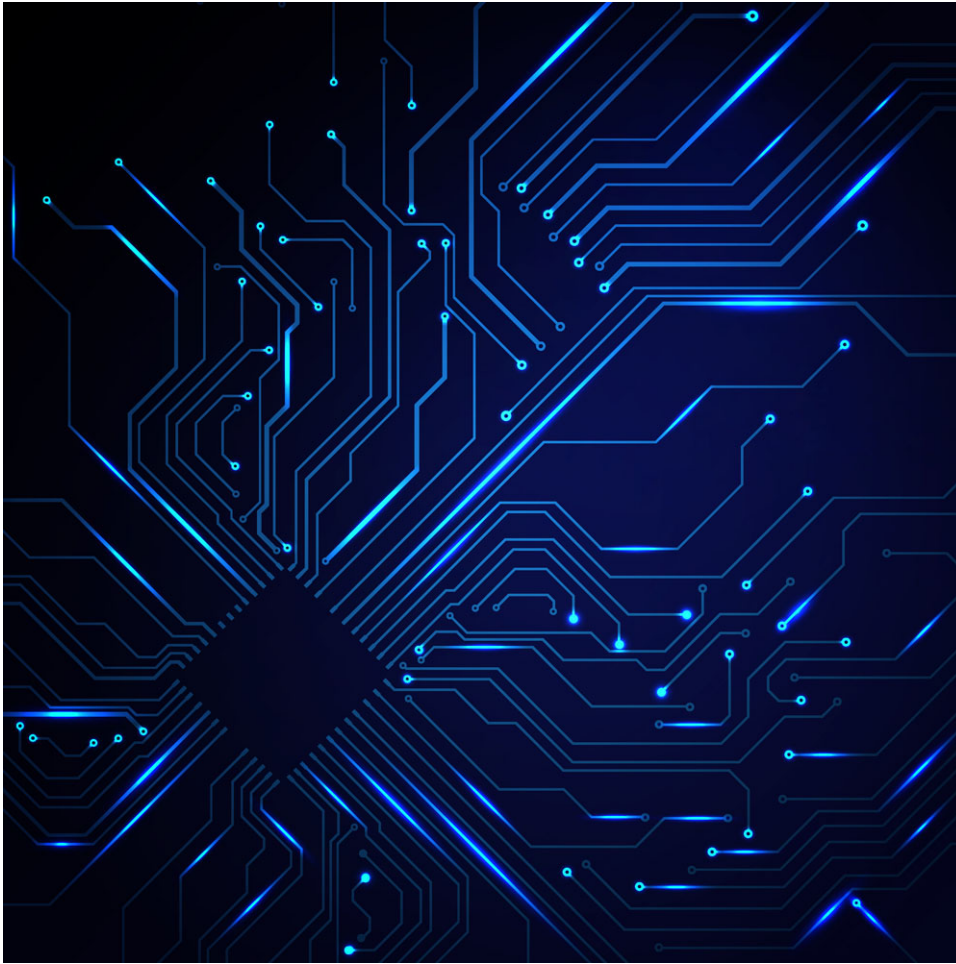




## LAB11

### Display Student ID & Birthday

Name: 黃翊宏      Student ID: 01357101

## Note

---

1. All files must be converted to PDF format before submission.
2. **Submission deadline:**  
**Next Tuesday at 1:00 PM**
3. Each person submits one assignment.

※ Please use Win + Shift + S for screenshots. Do not take photos of the screen with a phone.

## ★ Lab 1 :

### Main Code Screenshot

```
V studentID.sv X V birthday.sv
de0_empty > design > V studentID.sv
1  module studentID(
2      input clk, reset,
3      output logic [3:0]id
4  );
5      logic a, b, c, d, Da, Db, Dc, Dd;
6
7      dfflop dff1(.d(Da), .reset(reset), .clk(clk), .q(a));
8      dfflop dff2(.d(Db), .reset(reset), .clk(clk), .q(b));
9      dfflop dff3(.d(Dc), .reset(reset), .clk(clk), .q(c));
10     dfflop dff4(.d(Dd), .reset(reset), .clk(clk), .q(d));
11
12     assign Da = (b&c) | (c&(~d)) | ((~b)&(~c)&d);
13     assign Db = ((~a)&b) | ((~a)&c);
14     assign Dc = ((~c)&d) | (a&c);
15     assign Dd = ((~b)&c) | ((~a)&(~d)) | ((b)&(~c)&(d));
16
17     assign id[3:0] = {a, b, c, d};
18
19 endmodule
```

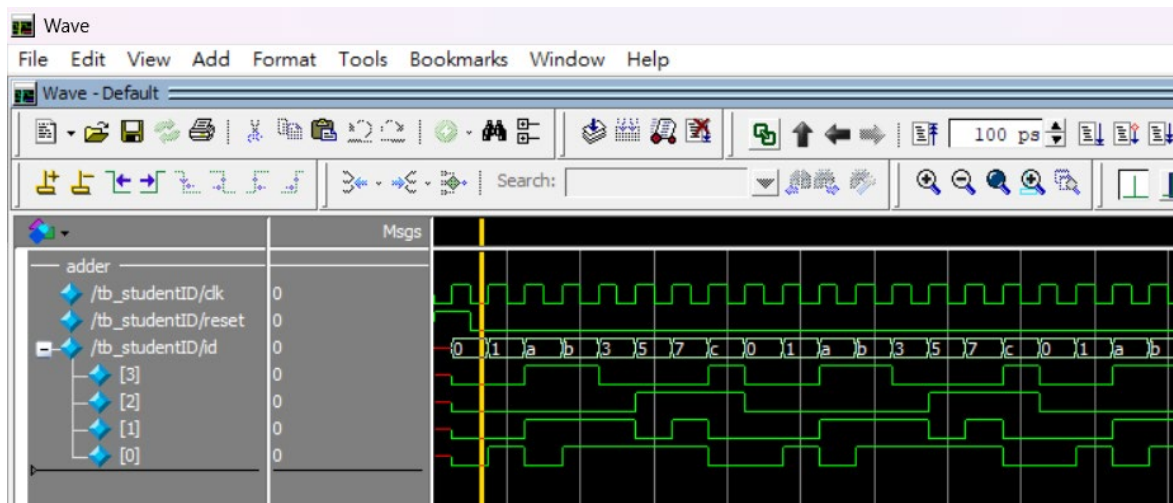
### Testbench Screenshot

```
V studentID.sv V tb_studentID.sv X V birthday.sv
de0_empty > simulation > tb > V tb_studentID.sv
1  module tb_studentID;
2      logic clk, reset;
3      logic [3:0]id;
4
5      studentID u_studentID(
6          .clk(clk),
7          .reset(reset),
8          .id(id)
9      );
10
11     always #5 clk = ~clk;
12     initial begin
13         clk = 0; reset = 1;
14         #10 reset = 0;
15         #500 $stop;
16     end
17 endmodule
18
```

## mcu.sv Code Screenshot

```
V studentID.sv  V mcu.sv  X  V birthday.sv
de0_empty > design > V mcu.sv
40  */
41  logic clk;
42  logic [3:0] id;
43
44  freg_div_23 freg(
45      .clk(CLK),
46      .reset(~BTN[0]),
47      .clk_out(clk)
48  );
49
50  //Lab 1
51  studentID id1(
52      .clk(clk),
53      .reset(~BTN[0]),
54      .id(id)
55  );
56
57  //Lab 2
58  /*birthday birth(
59      .clk(clk),
60      .reset(~BTN[0]),
61      .date(id)
62  );*/
63
64  ssd ssd1(
65      .a(id),
66      .seg(HEX0)
67  );
```

## Waveform and Result Screenshot (The values in the waveform must be clearly visible.)





## Lab 2 :

### Main Code Screenshot

```
V studentID.sv V birthday.sv X
de0_empty > design > V birthday.sv
1  module birthday(
2      input clk, reset,
3      output logic [3:0]date
4  );
5
6      logic a, b, c, d, Da, Db, Dc, Dd;
7
8      dfflipflop dff1(.d(Da), .reset(reset), .clk(clk), .q(a));
9      dfflipflop dff2(.d(Db), .reset(reset), .clk(clk), .q(b));
10     dfflipflop dff3(.d(Dc), .reset(reset), .clk(clk), .q(c));
11     dfflipflop dff4(.d(Dd), .reset(reset), .clk(clk), .q(d));
12
13     assign Da = ((~a) & (~c)) | ((a) & (~d));
14     assign Db = ((~a) & (~d)) | ((~a) & (~c)) | (a&c&d);
15     assign Dc = (~c) | ((~a)&(~b));
16     assign Dd = ((~a)&(~b)) | (a&c);
17
18     assign date[3:0] = {a, b, c, d};
19
20 endmodule
```

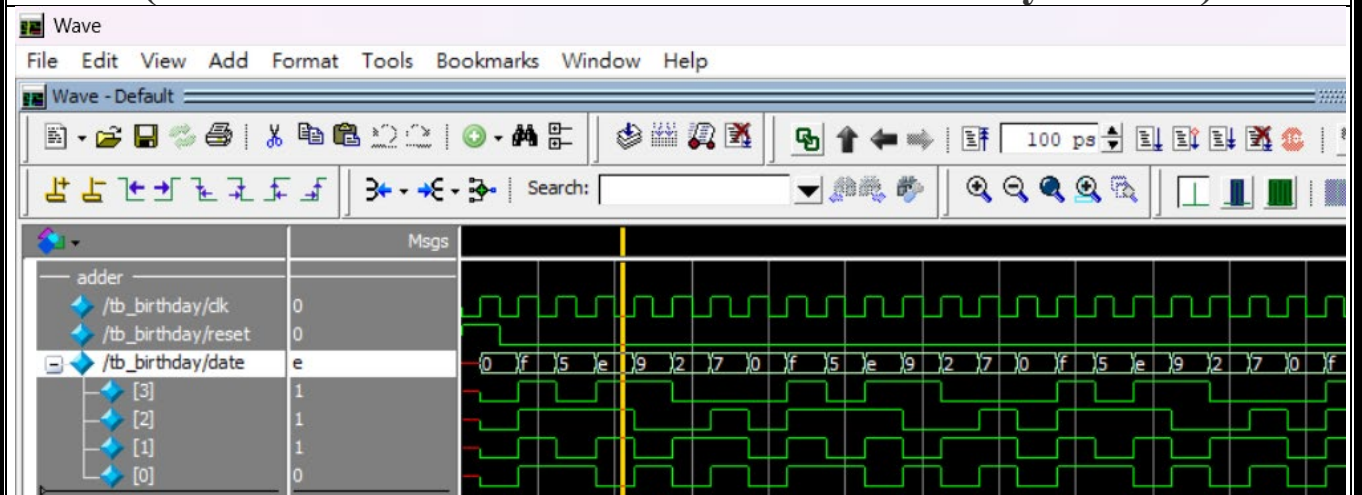
### Testbench Screenshot

```
V studentID.sv V birthday.sv V tb_birthday.sv X
de0_empty > simulation > tb > V tb_birthday.sv
1  module tb_birthday;
2      logic clk, reset;
3      logic [3:0]date;
4
5      birthday u_birthday(
6          .clk(clk),
7          .reset(reset),
8          .date(date)
9      );
10
11     always #5 clk = ~clk;
12     initial begin
13         clk = 0; reset = 1;
14         #10 reset = 0;
15         #500 $stop;
16     end
17 endmodule
18
```

## mcu.sv Code Screenshot

```
V studentID.sv V birthday.sv V mcu.sv X
de0_empty > design > V mcu.sv
41 logic clk;
42 logic [3:0] id;
43
44 freg_div_23 freg(
45     .clk(CLK),
46     .reset(~BTN[0]),
47     .clk_out(clk)
48 );
49
50 //Lab 1
51 /*studentID id1(
52     .clk(clk),
53     .reset(~BTN[0]),
54     .id(id)
55 );*/
56
57 //Lab 2
58 birthday birth(
59     .clk(clk),
60     .reset(~BTN[0]),
61     .date(id)
62 );
63
64 ssd ssd1(
65     .a(id),
66     .seg(HEX0)
67 );
68
```

## Waveform and Result Screenshot (The values in the waveform must be clearly visible.)





## Lab Summary and Reflection :

### Summary

In this week's experiment, we implemented on the Altera DE0 board a pair of finite-state machines in SystemVerilog to sequentially drive seven-segment displays with my own student ID (01357101) and my birthday. Following the lab requirements, I first studied the encoding rules: for the student ID, any digit that repeats must be replaced by successive letters A, B, C, ... except for its final occurrence; for the birthday, the first repeat stays as the digit itself and subsequent repeats are mapped in reverse order from F back toward A. I then translated these rules into state diagrams and corresponding state-transition tables. Using Karnaugh-map minimization, I derived simplified Boolean expressions for each D-flip-flop input in the unit and tens modules. Each module—one for the four-bit output digit along with reset-and-carry logic—was coded in its own SystemVerilog file and instantiated within a top-level mcu.sv. A testbench was created to verify that both the student ID "01357101" → "AB357C01" and the birthday encoding produced the expected waveforms. After successful simulation, I instantiated a frequency divider (freq\_div\_23) to slow down the onboard clock and mapped each 4-bit output to the appropriate HEX display pins. Final verification on the DE0 board showed the correct sequence of characters scrolling on the seven-segment displays .

### Reflection

Working through this lab reinforced my understanding of how abstract encoding rules translate into concrete FSM implementations. Crafting the state diagrams and truth tables deepened my appreciation for the methodical nature of synchronous design, while Karnaugh-map simplification underscored the practical benefits of logic minimization for FPGA resource efficiency. Coding separate modules for the units and tens digits enforced good design modularity, and writing a comprehensive testbench sharpened my simulation and debugging skills when waveform outputs did not immediately match expectations. Finally, deploying the design on real hardware—the DE0 development board—and observing the seven-segment display output provided valuable insight into timing considerations, pin-assignment constraints, and the end-to-end workflow of digital-logic projects. I feel more confident now in moving from HDL code to working hardware.

Good luck to me on the final exam next week!